

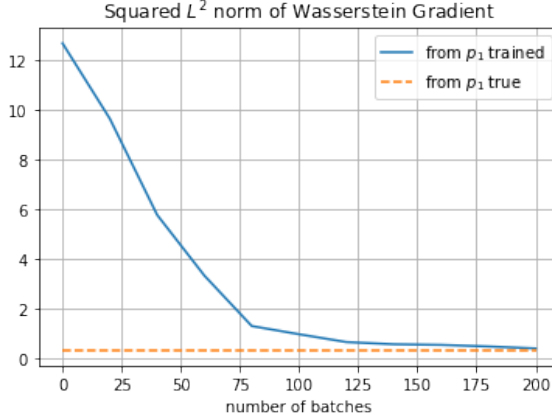


Figure A.1: Computed values of $\|\nabla_{\mathcal{W}_2} F_{n+1}(p_{n+1})\|_{p_{n+1}}^2$ from $N = 2000$ samples, where p_{n+1} is pushforwarded by a trained neural network transport T from a Gaussian initial p_n in $\mathbb{R}^2$, $n = 0$. The blue line shows the value as the training progresses, and the dashed line is a base value computed from the analytical solution p_{n+1}^{true} (where the Wasserstein gradient vanishes).

D Numerical evidence to support Assumption 1

We conduct training of one JKO block to verify (30) in Assumption 1. The code is available at https://github.com/yixintan-zeta/jko_wass_grad.

Data is in $\mathbb{R}^2$. Let $n = 0$, $p_0 = \mathcal{N}([3, 3]^T, I)$, $q = \mathcal{N}(0, I)$, and $G(\rho) = \text{KL}(\rho||q)$. The step size $\gamma = 0.5$. We train a JKO flow network block to minimize F_{n+1} in (24) via the training objective (25), where T is parametrized by a neural ODE block consisting of two hidden layers with 128 hidden dimensions and using the softplus activation ($\beta = 20$). We use 10,000 training samples, batch size 2000, and 200 total iterations, with learning rate 10^{-4} . The implementation of the JKO flow network follows the setup in [74].

Making use of the explicit expression of $\nabla_{\mathcal{W}_2} G(\rho)$, let ξ be ξ_1 , then (27) gives that

$$\xi = (\nabla V + \nabla \log p_1) - \frac{T_{p_1}^{p_0} - \text{Id}}{\gamma},$$

where $V(x) = \|x\|^2/2$. To compute ξ as a vector field, we used $N = 2000$ samples $x_i \sim p_0$, and then the neural-network trained T will push-forward x_i to $T(x_i) \sim p_1$. Let $X_0 = \{x_i^{(0)} := x_i\}_{i=1}^N$ and $X_1 = \{x_i^{(1)} := T(x_i)\}_{i=1}^N$ be two data clouds. We approximate the OT map $T_{p_1}^{p_0}$ evaluated on $x_i^{(i)}$ by the solution of a discrete OT problem, computed by the Python POT package [1]. The score function $\nabla \log p_1$ is approximately computed by a kernel approach, where we used a Gaussian kernel with a properly chosen bandwidth parameter. Once $\xi(x_i^{(1)})$ is computed at every sample, we can approximately compute the squared L^2 norm $\|\xi\|_{p_1}^2$ by a sample average.

We compute ξ at p_1 induced by trained T not only at the end of training but also during intermediate iterations. This will show the change in the Wasserstein gradient as the neural network training progresses. At 0, 20, ..., 200 batches, the estimated L^2 norm is shown in Figure A.1. The dash-line shows the computed value of the Wasserstein gradient at the true solution $p_1^{\text{true}} = \mathcal{N}([2, 2]^T, I)$, since the JKO step is from a Gaussian p_0 and thus the true population minimizer p_1 is analytically available. The numerical value is not exactly zero because it is also computed on $N = 2000$ finite samples. The dash-line shows a baseline of the numerical L^2 norm,

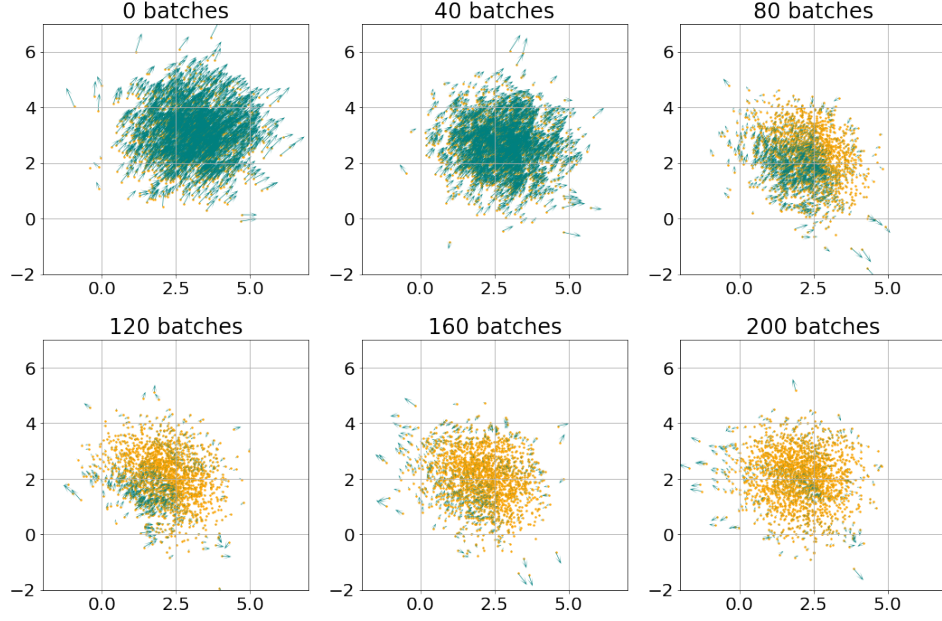


Figure A.2: The Wasserstein gradient vector field ξ at samples $x_i^{(1)} = T(x_i)$ (shown by green arrows), where T is the trained neural network transport map, plotted as the training progresses. The yellow dots are samples $x_i^{(1)}$. The length of the arrow is proportional to the magnitude of $\|\xi(x_i^{(1)})\|$.

and it can be seen that at the end of training, the neural network learned p_1 archives a comparable value.

We further illustrate the vector field ξ on $x_i^{(1)}$'s in Figure A.2, which shows the evolution of the Wasserstein gradient over training iterations. It can be seen that the magnitude of the vector field decreases, and the values are getting small at least within the region where the distribution density p_1 has a large value. In the outskirts, the vector field ξ does not numerically get small because it is at the tail of the Gaussian distribution, where a small L^2 norm does not imply pointwise smallness of ξ in these regions and the estimation of the vector field may also be of lower accuracy.